

WDT Project



Timeline Documentation

For Egy Metro Guide Project

Member name	ID
Abd Alrahman Saed Alsayed	202302377
Mahmoud Mohamed Hegazy	202302612
Mahmoud Fawzy Mahmoud	202302484
Ali Yasser Mohamed	202305666
Abdullah Ibrahim Gaballah	202302701

Supervised by: Dr Hossam Eladly

Cairo Metro Project — Timeline Documentation

1. Project Idea and Objective

The Cairo Metro project was initiated as a trip-planning and fare-calculation system built using ASP.NET Core MVC. The core objective is to model the Cairo Metro network as a graph and enable users to:

- Find the shortest route between two stations
- Calculate the number of stations and transfers
- Estimate travel time
- Determine fare based on pricing rules

From the beginning, the system was designed around a **data-first architecture**, where the metro network (lines, stations, and connections) is the foundation of all functionalities.

2. Solution Architecture Design

The project follows a **layered architecture** separating concerns across three main projects:

- Metro → ASP.NET Core MVC Web Application
- Metro.Core → Domain layer (entities, DTOs, interfaces)
- Metro.Data → Data access layer (EF Core, repositories, seeding)

This structure reflects clean separation between:

- Domain modeling
- Data persistence
- Presentation layer

Dependency Injection is used to connect repositories and services within the application pipeline.

3. Database Schema Setup

The database schema was established using **Entity Framework Core (Code First)** with SQL Server.

Key Milestone

- **March 3, 2026** → Initial migration: `InitialCreation`

This migration created the core tables:

- `Lines`
- `Stations`
- `StationConnections`
- `PricingRules`

Design decisions:

- All IDs are manually assigned (`ValueGeneratedNever`)
- Station ordering is preserved per line
- Connections are **directed edges** (graph modeling)
- Interchange stations are modeled as **separate rows per line**, connected via transfer edges

This schema forms the backbone of the routing engine.

4. Seed Data Preparation

The project uses a **hybrid seeding strategy**:

- JSON-based runtime seeding (primary approach)

Key Milestones

- **April 10, 2026** → Migration: SeedData
- Runtime seeder implemented (MetroDataSeeder)

Seed dataset includes:

- 3 metro lines
- 89 stations
- 182 directed connections
- 3 pricing rules

The seeder:

- Loads JSON files
- Inserts data in dependency order
- Is idempotent (prevents duplicates)

This ensures that the metro network is always available at application startup.

5. Route Algorithm Implementation

The routing logic is based on a **graph traversal approach**, specifically a **Dijkstra-like shortest path algorithm**.

Key modeling decisions:

- Stations = nodes
- Connections = edges
- Transfers = explicit cross-line edges

Algorithm responsibilities:

- Compute shortest path between stations
- Traverse both intra-line and transfer edges
- Return ordered station list

Although the domain and graph structure fully support routing, the implementation is still **partially integrated** into the application flow.

6. Pricing, Transfer, and Travel-Time Rules

Business rules were defined early in the project:

Pricing Rules

- 1–9 stations → 8 EGP
- 10–16 stations → 10 EGP
- 17+ stations → 15 EGP

Travel Time

- Fixed estimate: **2 minutes per station**

Transfers

- Derived from graph traversal across line-linked stations

These rules are encapsulated in the `PricingRule` entity and expected to be applied during route result calculation.

7. Repository and Service Layering

A repository pattern was implemented to abstract data access:

Interfaces:

- `ILineRepository`

- IStationRepository
- IStationConnectionRepository
- IPricingRuleRepository

Repositories:

- Wrap EF Core queries
- Provide read-only operations
- Include navigation properties where needed

8. Web UI Development

Key Milestone

- **April 29, 2026** → Web app import commit: 9d674f3

The MVC application was successfully integrated into the solution and has since evolved beyond the initial scaffolded state.

Current UI State

- Fully implemented MVC interface for metro functionality
- Station selection UI (dropdowns)
- Route result display including:
 - Station path
 - Transfers stations
 - Number of transfers
 - Number of stations
 - Estimated travel time
 - Calculated fare

The UI is now fully connected to backend logic and no longer in a scaffold-only state. It represents a completed **Phase 3 (MVC Integration)** implementation.

9. Testing Phase

Testing has progressed beyond initial groundwork and now includes:

- Coverage includes:
 - Data validation
 - Core business logic
 - Routing-related functionality (basic level)

Current Status

- Core logic is tested and functional
- Routing pipeline has been validated through tests

10. Refactor Toward MetroService

The refactoring effort toward a centralized service layer has been **successfully completed**.

Achievements

- MetroService implemented
- Business logic centralized into a single service layer

Responsibilities now encapsulated:

- Route calculation (Dijkstra-based)
- Transfer counting
- Fare calculation using PricingRules
- Travel time estimation
- Returning RouteResultDto

Outcome

- Clean separation between controller and business logic
- Controllers now act as orchestration endpoints only
- System is aligned with service-based architecture principles

11. Repository and Data Milestones

April 29, 2026

Includes:

- Domain entities
- DbContext
- Seed data (JSON + SQL support)
- Repository implementations

This established the **data and persistence foundation** of the system.

12. Current Status (As of May 1, 2026)

What is Completed

- Full metro graph modeled in database
- Seed data successfully loads at runtime
- Repository layer implemented
- Pricing rules defined and applied
- **Dijkstra-based routing fully implemented and wired**
- Transfer counting logic implemented
- Fare calculation pipeline completed
- Travel time computation integrated (2 minutes per station)
- MetroService fully implemented
- Complete routing pipeline returning RouteResultDto
- MVC UI fully functional and connected to backend
- Controller endpoints implemented for UI interaction
- Solution builds and runs successfully

What is Missing

- Production-level features and hardening

13. Current Gaps and Next Steps

Long-Term Direction

- **Map Integration**
 - Leaflet-based interactive map
 - Route visualization
- **GPS Features**
 - Detect user location
 - Find nearest station (Haversine formula)
- **API Layer Expansion**
 - Public endpoints for external use
- **Performance Optimization**
 - Caching (stations, routes)
 - Query optimization
- **Production Readiness**
 - Monitoring
 - Logging
 - Scalability improvements

Final Summary

As of May 1, 2026, the Cairo Metro project is **data-complete but feature-incomplete**.

The project has successfully transitioned from a **data-focused backend system** to a **fully functional metro trip planner**:

- Routing engine is implemented and operational
- Business logic is centralized in MetroService
- UI is complete and connected
- Core features (routing, pricing, time, transfers) are fully working

The project is now entering the **final stage**, focused on:

- API exposure

- Production readiness
- Advanced features (maps, GPS, performance)